

Introdução ao Caos - AA1

Sumário

1	Pontos fixos e retratos de fase	1
1a)	Sistema linear I	1
1b)	Sistema linear II	2
1c)	Sistema não linear	3
2	Pêndulo e aproximação para amplitudes moderadas	5
3	Mapa de grau 2	6
3a)	Expoente de Lyapunov nos regimes periódico e caótico	6
	<i>Séries temporais • Expoente de Lyapunov e sensibilidade às condições iniciais • Diagramas de cobweb</i>	
3b)	Rota para o caos por duplicação de período	8
	<i>Expoente de Lyapunov nas duplicações</i>	
3c)	Janelas periódicas imersas na região caótica	9
	<i>Janela de período 3 • Duplicação de período dentro da janela</i>	
4	Duplicações de período e constante de Feigenbaum	10
A	Rotinas utilizadas	11
	Ex. 1a) – ex1a.py	11
	Ex. 1b) – ex1b.py	11
	Ex. 1c) – ex1c.py	12
	Ex. 2 – ex2.py	13
	Ex. 3 e 3a) – ex3a.py	14
	Ex. 3b) – ex3b.py	16
	Ex. 3c)i) – ex3ci.py	18
	Ex. 3c)ii) – ex3cii.py	19
	Ex. 4 – ex4.py	22

1. Pontos fixos e retratos de fase

1a). Sistema linear I

Selecione o sistema linear

$$\begin{cases} \dot{x} = -3x \\ \dot{y} = 3x - 2y \end{cases}$$

Podemos escrever $\vec{\dot{x}} = M\vec{x}$, onde

$$\vec{x} = \begin{bmatrix} x \\ y \end{bmatrix}; \vec{\dot{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}; M = \begin{bmatrix} -3 & 0 \\ 3 & -2 \end{bmatrix}$$

Os pontos fixos satisfazem $\vec{\dot{x}} = 0$. Como $\det M \neq 0$, os pontos fixos são $x^* = y^* = 0$.

Pra avaliar a estabilidade dos pontos fixos, podemos calcular os autovalores de M através das raízes do polinômio característico

$$\det(M - \lambda I) = 0 \iff \begin{vmatrix} -3 - \lambda & 0 \\ 3 & -2 - \lambda \end{vmatrix} = 0 \Rightarrow (\lambda + 2)(\lambda + 3) = 0 \iff \begin{cases} \lambda = -2 \\ \lambda = -3 \end{cases}$$

Como ambos os autovalores são reais e negativos, o ponto fixo identificado é estável. Podemos determinar os autovetores associados a cada autovalor, também

- $\lambda = -2$:

$$(M + 2I)\vec{v}_{-2} = 0 \iff \begin{bmatrix} -1 & 0 \\ 3 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = 0 \iff a = 0, b \in \mathbb{R}_*$$

Podemos selecionar, por conveniência,

$$\vec{v}_{-2} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- $\lambda = -3$:

Similarmente, def. $\vec{v}_{-3} = \begin{bmatrix} u \\ v \end{bmatrix}^T$, temos $(M + 3I)\vec{v}_{-3} = 0 \iff$

$$\iff \begin{bmatrix} 0 & 0 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = 0 \iff v = -3u$$

Para $u = 1$, $v = -3$, e uma escolha possível é

$$\vec{v}_{-3} = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$$

Por meio das equações do sistema, podemos gerar soluções gerais para as EDOs. Para x , $x(t) = x_0 e^{-3t}$, então $\dot{y} + 2y = 3x_0 e^{-3t}$. Tomando $y = y_h + y_p$, podemos resolver a EDO homogênea

$$\dot{y}_h + 2y_h = 0 \iff y_h = y_0 e^{-2t}$$

Para a solução particular, considere $y_p = K e^{-3t}$, e

$$\dot{y}_p + 2y_p = 3x_0 e^{-3t} = -3K e^{-3t} + 2K e^{-3t} = 3x_0 e^{-3t} \longrightarrow -3K + 2K = 3x_0 \iff K = -3x_0$$

Ou seja,

$$y(t) = (y_0 + 3x_0)e^{-2t} - 3x_0 e^{-3t} = x_0 e^{-3t} \cdot \vec{v}_{-3} + (y_0 + 3x_0)e^{-2t} \cdot \vec{v}_{-2}$$

Com isso, podemos esboçar o retrato de fase desse sistema, como na fig. 1.

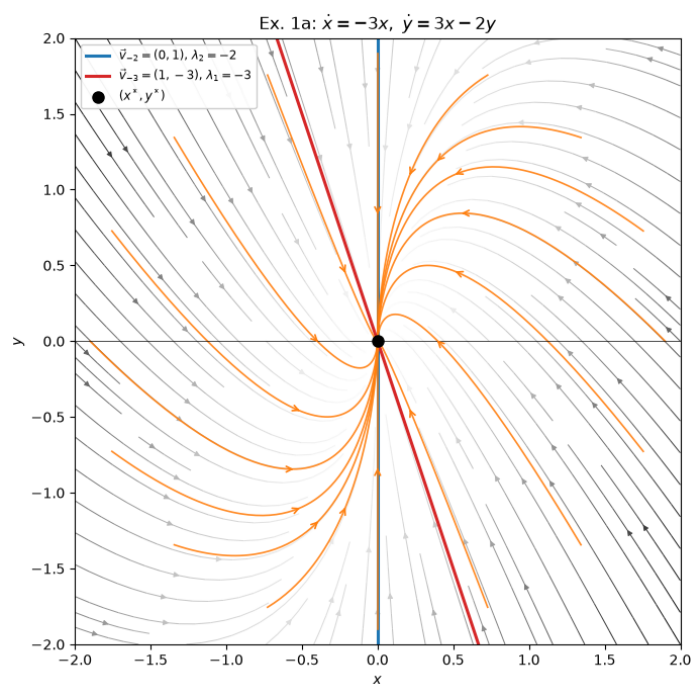


Figura 1. Retrato de fase do sistema $\dot{x} = -3x$, $\dot{y} = 3x - 2y$.

1b). Sistema linear II

Escolhendo

$$\begin{cases} \dot{x} = -x + y \\ \dot{y} = -4x + y \end{cases}$$

Podemos realizar o mesmo tratamento que previamente. Primeiro, tomando

$$\vec{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \vec{\dot{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}, M = \begin{bmatrix} -1 & 1 \\ -4 & 1 \end{bmatrix}$$

Para termos $\vec{\dot{x}} = 0$, devemos ter $x = y$, e $4x = y \iff x = y = 0$.

Os autovalores associados a M serão as raízes do polinômio característico

$$\begin{vmatrix} -1-\lambda & 1 \\ -4 & 1-\lambda \end{vmatrix} = 0 \iff \lambda^2 + 3 = 0 \iff \lambda_{\pm} = \pm i\sqrt{3}$$

Como a parte real dos autovalores é zero, eles apresentam rotação, mas com curvas fechadas, no espaço de fases. Se avaliarmos o sistema nos pontos $(1, 0)$, $(0, 1)$, teremos

$$\begin{cases} \dot{x}(1, 0) = -1 \\ \dot{y}(1, 0) = -4 \\ \dot{x}(0, 1) = 1 \\ \dot{y}(0, 1) = 1 \end{cases}$$

Ou seja, estamos indo $[-1 \quad -4] \rightarrow [1 \quad 1]$, ou seja, a rotação é no sentido horário. De qualquer forma, o ponto fixo é um centro, ou seja, ainda que estável, não é assintoticamente estável.

Podemos determinar a solução explícita considerando as EDOs fornecidas. Para $\dot{x}$, podemos fazer

$$\ddot{x} = -\dot{x} + \dot{y} = -\dot{x} - 4x + y$$

Mas, $y = \dot{x} + x$, então

$$\ddot{x} = -3x \iff x(t) = x_0 \cos \omega t + \frac{y_0 - x_0}{\omega} \cdot \sin \omega t$$

considerando o estado inicial como (x_0, y_0) . A partir disso, $y(t) = \dot{x} + x$.

Com isso, podemos esboçar o retrato de fase do sistema, como na fig. 2.

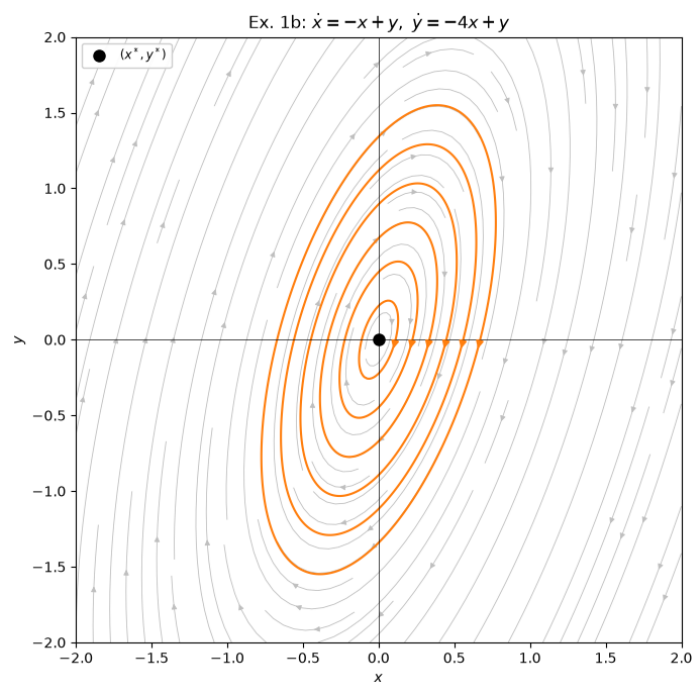


Figura 2. Retrato de fase do sistema escolhido.

1c). Sistema não linear

Os pontos fixos ocorrerão quando $(\dot{x}, \dot{y}) = 0 \iff$

$$\begin{cases} x = 0, \text{ ou } x + 2y = 3 \\ y = 0, \text{ ou } x + y = 2 \end{cases} \begin{cases} x = y = 0 \Rightarrow (x^*, y^*) = (0, 0) \\ x = 0, x + y = 2 \Rightarrow (x^*, y^*) = (0, 2) \\ y = 0, x + 2y = 3 \Rightarrow (x^*, y^*) = (3, 0) \\ x + 2y = 3, x + y = 2 \Rightarrow \begin{cases} x^* + 2y^* = 3 \\ x^* + y^* = 2 \end{cases} \iff (x^*, y^*) = (1, 1) \end{cases}$$

Denominando $f(x, y) = \dot{x}$, $g(x, y) = \dot{y}$, podemos escrever a jacobiana

$$J(x, y) = \begin{bmatrix} \partial_x f & \partial_y f \\ \partial_x g & \partial_y g \end{bmatrix} = \begin{bmatrix} 3 - 2x - 2y & -2x \\ -y & 2 - x - 2y \end{bmatrix}$$

Avaliando em cada ponto, teremos

- $(0, 0)$:

$$J = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \Rightarrow \begin{cases} \lambda = 3 \\ \lambda = 2 \end{cases}$$

Como ambos são positivos e reais, o ponto fixo é instável.

- (0, 2):

$$J = \begin{bmatrix} -1 & 0 \\ -2 & -2 \end{bmatrix}$$

Disso, podemos avaliar o polinômio característico associado a essa jacobiana

$$\begin{vmatrix} -1-\lambda & 0 \\ -2 & -2-\lambda \end{vmatrix} = 0 \iff \begin{cases} \lambda = -1 \\ \lambda = -2 \end{cases}$$

Como ambos são negativos e reais, o ponto fixo é estável.

- (3, 0):

$$J = \begin{bmatrix} -3 & -6 \\ 0 & -1 \end{bmatrix}$$

Disso,

$$\begin{vmatrix} -3-\lambda & -6 \\ 0 & -1-\lambda \end{vmatrix} = 0 \iff \begin{cases} \lambda = -3 \\ \lambda = -1 \end{cases}$$

Como ambos são reais e negativos, o ponto fixo é estável.

- (1, 1):

$$J = \begin{bmatrix} -1 & -2 \\ -1 & -1 \end{bmatrix}$$

Disso,

$$\begin{vmatrix} -1-\lambda & -2 \\ -1 & -1-\lambda \end{vmatrix} = 0 \iff (1+\lambda)^2 = 2 \iff \lambda_{\pm} = -1 \pm \sqrt{2}$$

Nesse caso, como temos um autovalor positivo e um negativo, e ambos reais, temos um ponto de sela.

Podemos esboçar o espaço de fase a partir das EDOs, sem uma solução geral, tal qual a fig. 3

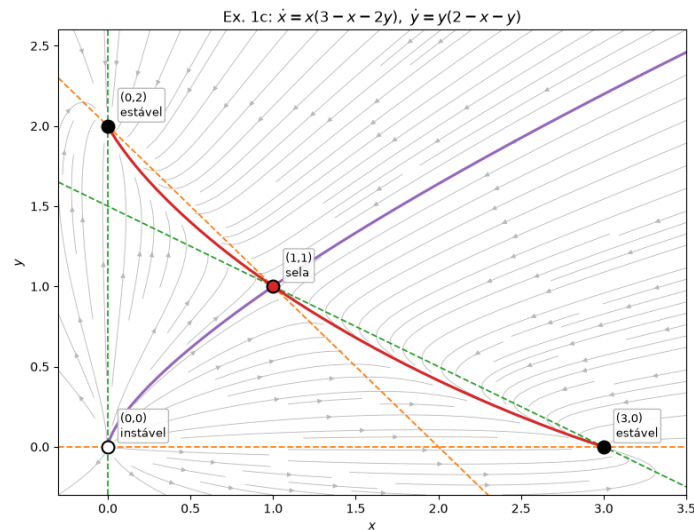


Figura 3. Retrato de fase do sistema em questão.

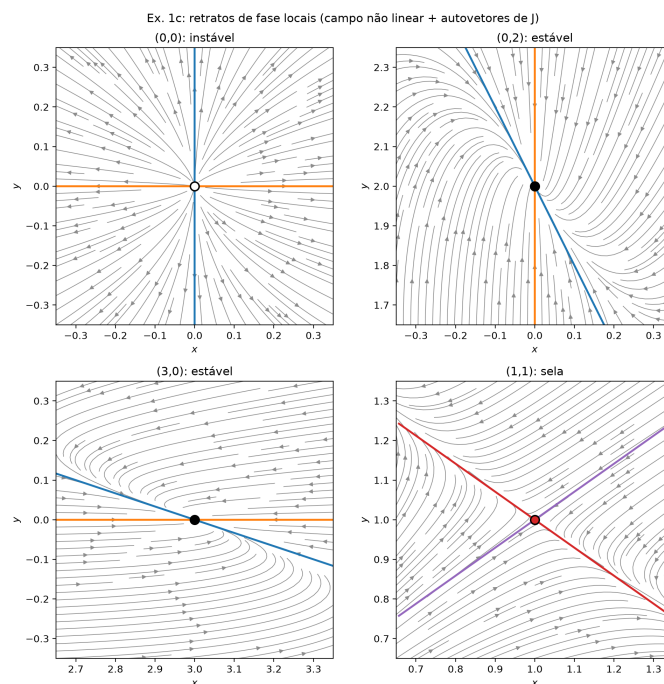


Figura 4. Retratos de fase locais, para cada ponto fixo identificado.

2. Pêndulo e aproximação para amplitudes moderadas

Definindo $f(\theta) = -\theta + \frac{\theta^3}{6}$, os pontos fixos serão em $f(\theta^*) = 0$, para algum θ^* . Para isso, devemos ter

$$\theta^*(-1 + (\theta^*)^2/6) = 0 \iff \begin{cases} \theta^* = 0 \\ \theta^* = \pm\sqrt{6} \end{cases}$$

A jacobiana é, então,

$$J = \begin{bmatrix} 0 & 1 \\ f'(\theta^*) & 0 \end{bmatrix}, \text{ onde } f'(\theta) = -1 + \frac{\theta^2}{2}$$

Assim, as raízes do polinômio característico são

$$\begin{vmatrix} -\lambda & 1 \\ f'(\theta^*) & -\lambda \end{vmatrix} = 0 \iff \lambda^2 = f'(\theta^*)$$

Assim, para

- $(0, 0)$: $f'(\theta^*) = -1$, então $\lambda = \pm i$. Como esses autovalores são imaginários puros, as órbitas são fechadas, e o ponto fixo é um centro.
- $(\pm\sqrt{6}, 0)$: $f'(\theta^*) = 2 \iff \lambda = \pm\sqrt{2}$. Como temos um autovalor positivo e um negativo, os pontos fixos são selas.

Se considerarmos a equação adimensionalizada original, podemos definir $g(\theta) = -\sin \theta$. Os pontos fixos são em $\sin \theta^* = 0$, ou seja

$$\theta^* = \pi n, n \in \mathbb{Z}$$

Podemos escrever a série de Taylor

$$\sin \theta = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} \theta^{2n+1} = \theta - \frac{1}{6}\theta^3 + \mathcal{O}(\theta^5)$$

De tal modo que, para pequenos ângulos tenhamos, de fato

$$\sin \theta \cong \theta - \frac{\theta^3}{6} \Rightarrow \ddot{\theta} = -\sin \theta \cong -\theta + \frac{1}{6}\theta^3$$

Podemos analisar esse mesmo sistema sob a perspectiva da energia se definirmos as energias potenciais

$$V_f'(\theta) = -f(\theta) \iff V_f(\theta) = \frac{\theta^2}{2} - \frac{\theta^4}{24}$$

$$V_g(\theta) = 1 - \cos \theta$$

assumindo $V(0) = 0$ para ambos os casos. Desse modo,

$$\ddot{\theta} = -V'(\theta) \Rightarrow \ddot{\theta}\dot{\theta} + \dot{\theta}V' = 0 \Rightarrow \frac{d}{dt} \left[\frac{\dot{\theta}^2}{2} + V(\theta) \right] = 0$$

Assim, como o sistema é conservativo, podemos definir $E := \frac{\dot{\theta}^2}{2} + V(\theta) = \text{cte}$. Disso, no ponto de retorno, $E = V(A)$, onde A é a amplitude de oscilação, e disso

$$\dot{\theta} = \pm \sqrt{2(V(A) - V(\theta))} \Rightarrow T(A) = 4 \int_0^A \frac{d\theta}{\sqrt{2[V(A) - V(\theta)]}}$$

Para g , é conhecido que $T_g(A) = 4K(k)$, $k = \sin(A/2)$. Para f , podemos escrever

$$T_f(A) = 4 \int_0^{\pi/2} \frac{d\varphi}{\sqrt{1 - \frac{A^2}{12}(1 + \sin^2 \varphi)}}$$

De modo que podemos expandir

$$T_g = 2\pi \left(1 + \frac{A^2}{16} + \frac{11A^4}{3072} + \dots \right)$$

$$T_f = 2\pi \left(1 + \frac{A^2}{16} + \frac{19A^4}{3072} + \dots \right)$$

A diferença entre as duas equações, então, se dá, primeiramente, no termo de ordem $\mathcal{O}(A^4)$. Para pequenas amplitudes, portanto, a aproximação, de fato, vale, sob uma perspectiva de período, também.

Por fim, podemos analisar o retrato de fase, e a diferença visual entre os dois períodos, na fig. 5.

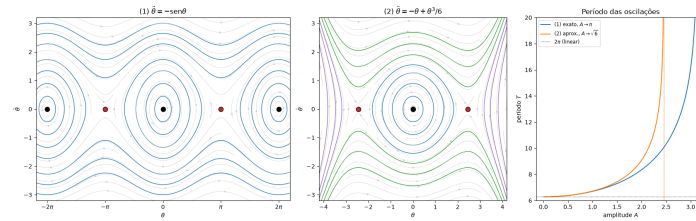


Figura 5. Retratos de fase e comparação dos períodos.

3. Mapa de grau 2

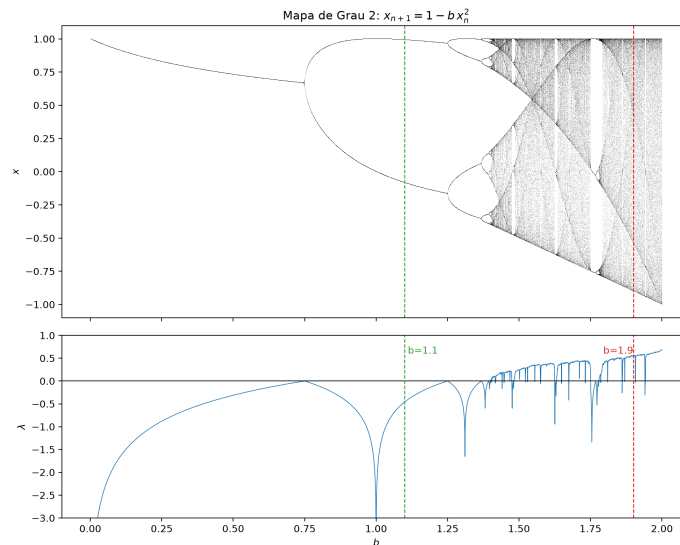


Figura 6. Mapa de bifurcações para $0 < b \leq 2$.

3a). Expoente de Lyapunov nos regimes periódico e caótico

A fig. 6 apresenta o diagrama de bifurcação para $0 < b \leq 2$ e, abaixo, o expoente de Lyapunov $\lambda(b)$. Nas regiões periódicas, $\lambda < 0$, e $\lambda = 0$ nos pontos de bifurcação ($b = 3/4$, $b = 5/4$, ...); nas regiões caóticas, onde o atrator preenche intervalos de x , $\lambda > 0$.

3a)i). Séries temporais

Os pontos fixos ocorrerão em $x^* = f(x^*) \iff bx^{*2} + x^* - 1 = 0 \Rightarrow x^* = \frac{-1 \pm \sqrt{1+4b}}{2b}$, para $x^* \in [-1, 1]$. Assim, como $f'(x) = -2bx$, o mapa perde estabilidade quando $|f'(x^*)| > 1$, ou seja, $f'(x^*) = 1 - \sqrt{1+4b}$, mas, $f'(x^*) < 1 \forall b \in (0, 2]$, então, a única fronteira que pode ser cruzada pelo mapa é em -1 , que ocorre em $b = 3/4$. Para $3/4 < b < 5/4$, o ciclo de período 2 tem multiplicador $\mu_2 = f'(x_+)f'(x_-) = 4(1-b)$, com $|\mu_2| < 1$, e é estável.

Considerando $x_0 = 0, 1$, podemos esboçar as séries temporais para dois valores de b , 1, 1 e 1, 9, como na fig. 7.

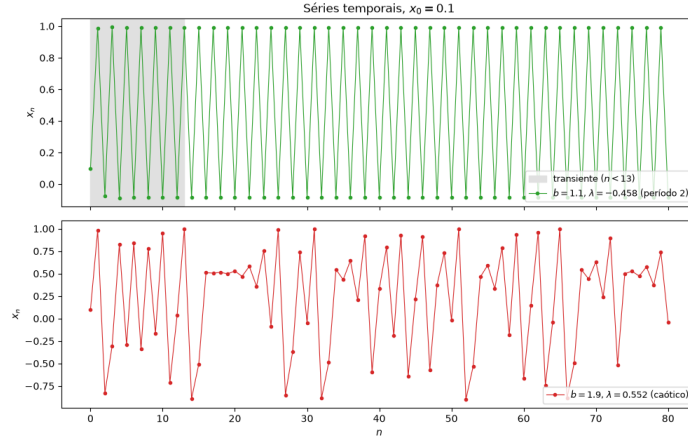


Figura 7. Série temporal do mapa de grau 2.

A região sombreada corresponde ao transiente da série periódica ($n < 13$); no caso caótico, $x_0 = 0,1$ já pertence ao atrator $[1-b, 1]$, e não há transiente.

3a)ii). Expoente de Lyapunov e sensibilidade às condições iniciais

Se considerarmos duas condições iniciais próximas, x_0 e $x'_0 = x_0 + \delta_0$, para algum $|\delta_0| \ll 1$. Denominando $\delta_n := x'_n - x_n$ como a separação entre duas órbitas, podemos expandir um mapa $f(x)$ em torno de x_n t.q.

$$x'_{n+1} = f(x_n + \delta_n) = f(x_n) + f'(x_n)\delta_n + \mathcal{O}(\delta_n^2) \Rightarrow \delta_{n+1} \cong f'(x_n)\delta_n$$

Assim, se aplicarmos $\delta_1 \cong f'(x_0)\delta_0$, e $\delta_2 \cong f'(x_1)\delta_1 = f'(x_1)f'(x_0)\delta_0$, podemos, por indução, escrever

$$\delta_{n+1} = \delta_0 \prod_{k=0}^n f'(x_k)$$

Se escrevermos o produto na forma $|\delta_n| = |\delta_0|e^{\lambda n}$,

$$\lambda = \lim_{n \rightarrow \infty} \frac{1}{n} \ln \left| \frac{\delta_n}{\delta_0} \right| = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=0}^{n-1} \ln |f'(x_k)|$$

que é o chamado expoente de Lyapunov. Com isso, conseguimos ter uma ideia de uma taxa média de separação por iteração passada, pensando computacionalmente. Para $\lambda < 0$, órbitas vizinhas convergem para uma mesma órbita, mas, para $\lambda > 0$, as perturbações crescem exponencialmente, que é uma representação da sensibilidade às condições iniciais. Por exemplo, para o mapa de grau 2, podemos analisar esse distanciamento, como na fig. 8. Nela, podemos ver que, para o sistema em regime caótico, as séries são praticamente indistinguíveis até $n \cong 35$, a partir do qual as trajetórias se distanciam significativamente. Isso pode, também, ser visualizado por meio do log-plot de $|\delta_n|$.

O contrário também vale para o mapa com $b = 1, 1$ e $\lambda = -0,46$, em que $|\delta_n|$ decai com $e^{\lambda n}$, de modo que as duas séries aparentem estarem sobrepostas durante todas as iterações.

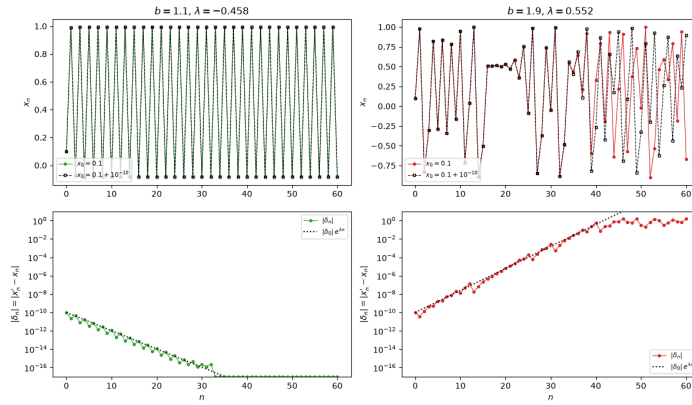


Figura 8. Visualização da sensibilidade às condições iniciais.

3a)iii). Diagramas de cobweb

Podemos construir o diagrama de cobweb a partir de $x_{n+1} = 1 - bx_n^2$, e, a partir de $(x_0, 0)$, teremos, primeiro, um movimento vertical até a curva, mapeando $(x_n, f(x_n)) \rightarrow (x_n, x_{n+1})$, e um horizontal até a diagonal. Para ambos os casos discutidos previamente, temos os mapas da fig. 9.

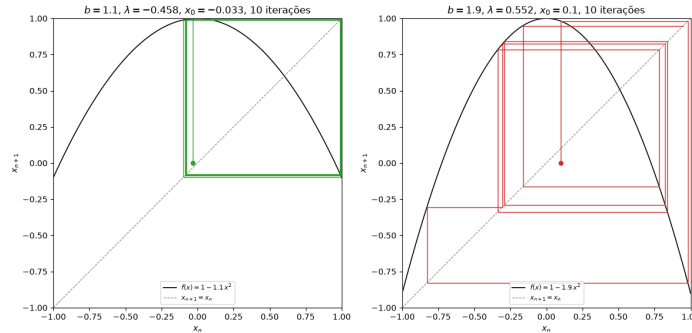


Figura 9. Diagrama de cobweb de ambos os casos, $b = 1, 1$ e $b = 1, 9$.

Para $b = 1, 1$ ($\lambda = -0,458$), a teia converge para um retângulo fechado com vértices em $x_{\pm}$, o atrator de período 2. Para $b = 1, 9$ ($\lambda = 0,552$), a teia não se fecha, e as iterações percorrem o intervalo $[1 - b, 1]$ sem se repetir.

3b). Rota para o caos por duplicação de período

3b)i). Expoente de Lyapunov nas duplicações

Se $\{x_j\}_{j=1}^p$ for um ciclo estável de período p , sobre ele, a soma que define o expoente de Lyapunov se repete a cada p passos. Se definirmos

$$\mu := \prod_{i=1}^p f'(x_i)$$

temos

$$\lambda = \frac{1}{p} \ln |\mu|$$

O ciclo é estável quando $|\mu| < 1 \Rightarrow \lambda < 0$. No ponto da duplicação, $|\mu| = 1 \Leftrightarrow \lambda = 0$. Assim, em cada duplicação do período, o expoente de Lyapunov vai a zero.

Podemos calcular numericamente os b 's em que isso acontece por meio do método de Newton aplicado às duplicações $f^{2^k}(x) - x$, e depois o método de Brent para solucionar $\mu_{2^k}(b) = -1$. Para as primeiras duas duplicações, teríamos $b_1 = 3/4$, como mencionado previamente, e $b_2 = 5/4$ de $\mu_2 = 4(1 - b) = -1$.

Isso pode ser visualizado a partir de um diagrama de bifurcação, como na fig. 10.

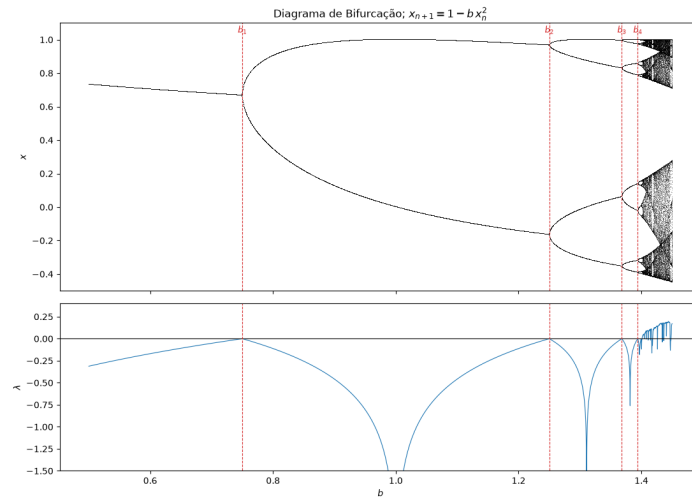


Figura 10. Diagrama de bifurcação do mapa quadrático.

3c). Janelas periódicas imersas na região caótica

3c)i). Janela de período 3

Podemos encontrar as janelas periódicas considerando que um período pode ser definido como, após o transiente, o menor p t.q. $|x_{n+p} - x_n| < 10^{-6}$, e colocando um máximo $p \leq 64$. Disso, se varreremos alguns valores de b , é possível encontrar a janela 1, $75 < b < \sim 1,77$ para o período 3. Disso, podemos dar um zoom na janela em questão, como na fig. 11.

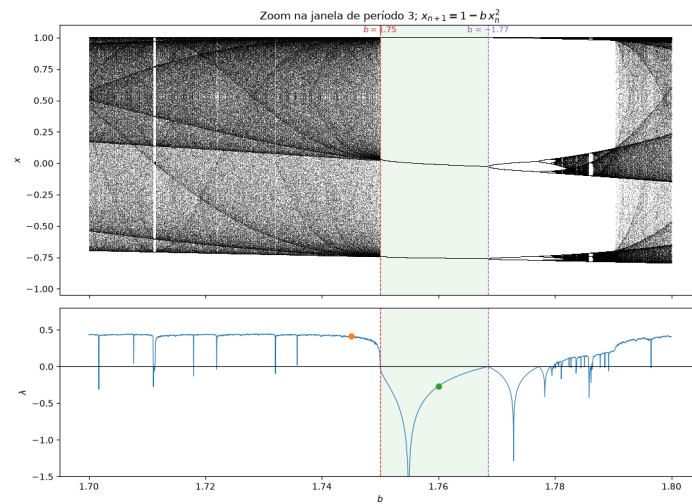


Figura 11. Zoom na janela de período 3.

Podemos, também, visualizar as séries temporais correspondentes aos valores de b identificados nos pontos da imagem na fig. 12.

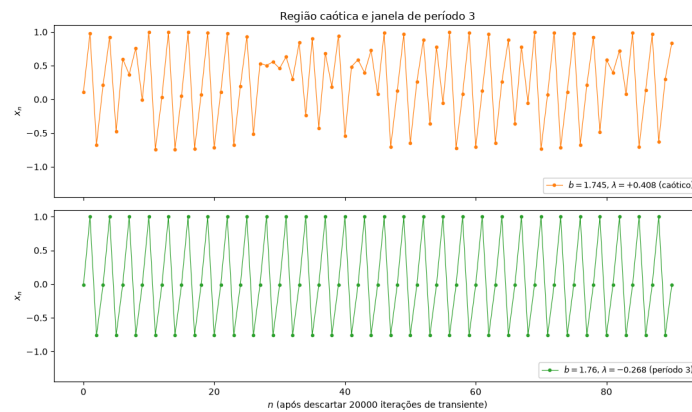


Figura 12. Séries temporais correspondentes a dois parâmetros de controle identificados na imagem.

3c)ii). Duplicação de período dentro da janela

Do item anterior, o ciclo de período 3 deixa de ser estável em $b \sim 1,77^1$, então, pelo mapa de bifurcação, conseguimos ver que haverá uma separação nesse ponto. Selecionando o intervalo $1,76853 \leq b \leq 1,790$, podemos utilizar o mesmo critério que em 3)c)i) para encontrar as duplicações e determinar a evolução ao caos.

Para evitar que as duplicações fiquem comprimidas, são feitos zooms sucessivos, cada um de b_k até um pouco depois de b_∞ , regime caótico. Como cada órbita de período p percorre p ramos, x_n é marcado a cada p iterações, e cada subplot mostra o ramo mais próximo de $x = 0$, por continuidade, com o eixo vertical sucessivamente ajustado.

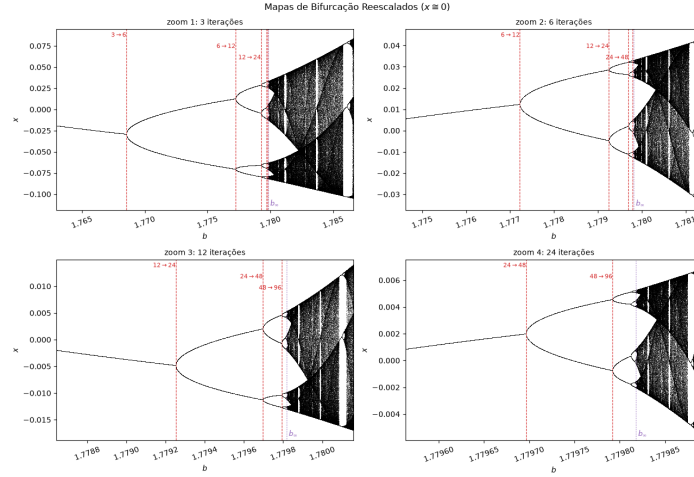


Figura 13. Mapas de bifurcação sucessivamente ampliados.

Podemos observar a rota ao caos considerando o distanciamento de trajetórias próximas para alguns valores do parâmetro de controle no intervalo determinado, como na fig. 14.

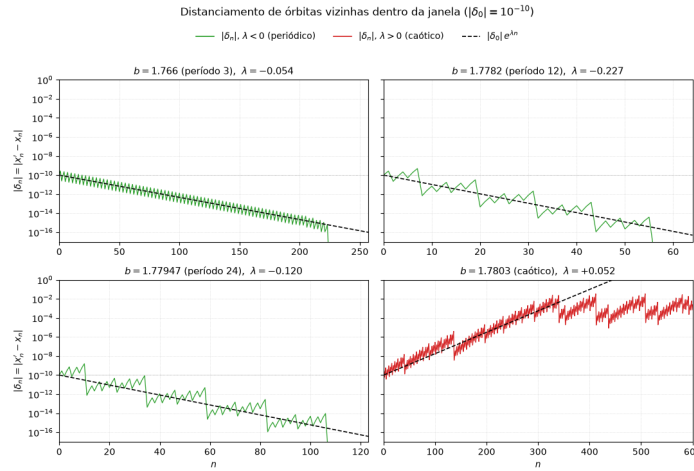


Figura 14. Distanciamento de trajetórias próximas para 4 valores de b .

4. Duplicações de período e constante de Feigenbaum

Os parâmetros a_n são obtidos considerando que, para o ciclo de período $p = 2^{n-1}$, o método de Newton aplicado a $f^p(x) - x$ fornece o ciclo, e o método de Brent resolve $\mu(a) = \prod_{i=1}^p f'(x_i) = -1$.

A primeira duplicação é exata tanto no mapa fornecido quanto no mapa logístico visto que, para o mapa de grau 2, $f'(x^*) = 1 - \sqrt{1 + 4b} = -1 \iff b_1 = 3/4$, como visto previamente.

¹O valor com mais casas ($b = 1,76853$) foi utilizado nas contas, mas a ponta do intervalo foi mostrada dessa forma por brevidade.

Tabela 1. Duplicações de período para o mapa $f(x) = 1 - bx^2$.

Períodos envolvidos na bifurcação	b_n	$D_n = b_n - b_{n-1}$	$\delta_n = D_{n-1}/D_n$
$1 \Rightarrow 2$	3/4 (exato)	—	—
$2 \Rightarrow 4$	5/4 (exato)	0,5 (exato)	—
$4 \Rightarrow 8$	1,36809894	0,11809894	4,2337
$8 \Rightarrow 16$	1,39404616	0,02594722	4,5515
$16 \Rightarrow 32$	1,39963124	0,00558508	4,6458
$32 \Rightarrow 64$	1,40082874	0,00119750	4,6639
$64 \Rightarrow 128$	1,40108527	0,00025653	4,6681

No mapa em questão, δ_n tende à constante de Feigenbaum (4,6692 ...).

A. Rotinas utilizadas

As rotinas abaixo geram as figuras e a tabela apresentadas (as imagens são salvas na pasta `imagens/`) e também estão disponíveis no [repositório da atividade](#).

Ex. 1a) – ex1a.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
6 os.makedirs(IMG_DIR, exist_ok=True)
7
8 A = np.array([[-3.0, 0.0],
9               [3.0, -2.0]]) # inicializando sistema
10
11
12 def campo(x, y):
13     return A[0, 0] * x + A[0, 1] * y, A[1, 0] * x + A[1, 1] * y # def. func campo
14
15
16 def solucao(x0, y0, t): # sol geral da EDO
17     x = x0 * np.exp(-3 * t)
18     y = (y0 + 3 * x0) * np.exp(-2 * t) - 3 * x0 * np.exp(-3 * t)
19     return x, y
20
21 lam, V = np.linalg.eig(A) # autovals
22 print("Autovalores:", lam)
23 print("Autovetores (colunas, normalizados):\n", V)
24 print("tr A =", np.trace(A), "| det A =", np.linalg.det(A),
25       "| tr^2 - 4 det =", np.trace(A) ** 2 - 4 * np.linalg.det(A)) # check tr, det
26
27 L = 2.0
28 fig, ax = plt.subplots(figsize=(7, 7)) # plot, etc
29
30 xg, yg = np.meshgrid(np.linspace(-L, L, 300), np.linspace(-L, L, 300))
31 u, v = campo(xg, yg)
32 ax.streamplot(xg, yg, u, v, color=np.hypot(u, v), cmap="Greys",
33              density=1.3, linewidth=0.7, arrowsize=0.8)
34
35 s = np.linspace(-L, L, 2)
36 ax.plot(0 * s, s, color="tab:blue", lw=2.2,
37        label=r"$\mathbf{v}_2=(0,1)$, $\lambda_2=-2$ (lenta)")
38 ax.plot(s, -3 * s, color="tab:red", lw=2.2,
39        label=r"$\mathbf{v}_1=(1,-3)$, $\lambda_1=-3$ (rápida)")
40
41 t = np.linspace(0, 6, 600)
42 theta = np.linspace(0, 2 * np.pi, 16, endpoint=False)
43 for th in theta:
44     x0, y0 = 1.9 * np.cos(th), 1.9 * np.sin(th)
45     xs, ys = solucao(x0, y0, t)
46     ax.plot(xs, ys, color="tab:orange", lw=1.2, alpha=0.9)
47
48 k = 40
49 ax.annotate("", xy=(xs[k + 1], ys[k + 1]), xytext=(xs[k], ys[k]),
50            arrowprops=dict(arrowstyle="->", color="tab:orange", lw=1.2))
51
52 ax.plot(0, 0, "k", ms=8, zorder=5, label="ponto fixo")
53
54 ax.set_xlim(-L, L)
55 ax.set_ylim(-L, L)
56 ax.set_aspect("equal")
57 ax.axhline(0, color="k", lw=0.5)
58 ax.set_xlabel("$x$")
59 ax.set_ylabel("$y$")
60 ax.set_title(r"Ex. 1a: $\dot{x}=-3x, \dot{y}=3x-2y$")
61 ax.legend(loc="upper left", fontsize=8.5, framealpha=0.95)
62 plt.tight_layout()
63 plt.savefig(os.path.join(IMG_DIR, "retrato_fase_1a.png"), dpi=200)
64 plt.show()

```

Ex. 1b) – ex1b.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
6 os.makedirs(IMG_DIR, exist_ok=True)
7

```

```

8  A = np.array([[ -1.0, 1.0],
9               [-4.0, 1.0]]) # mesmo procedimento do 1a
10 w = np.sqrt(3.0)
11
12
13 def campo(x, y):
14     return A[0, 0] * x + A[0, 1] * y, A[1, 0] * x + A[1, 1] * y
15
16
17 def H(x, y):
18     return 4 * x**2 - 2 * x * y + y**2
19
20
21 def solucao(x0, y0, t):
22     xd0 = -x0 + y0
23     x = x0 * np.cos(w * t) + (xd0 / w) * np.sin(w * t)
24     xd = -x0 * w * np.sin(w * t) + xd0 * np.cos(w * t)
25     y = xd + x
26     return x, y
27
28 lam = np.linalg.eigvals(A)
29 print("Autovalores:", lam)
30 print("tr A =", np.trace(A), "| det A =", round(np.linalg.det(A), 12))
31
32 t = np.linspace(0, 2 * np.pi / w, 2001)
33 x, y = solucao(0.5, 0.3, t)
34 print("Variacao relativa de H ao longo de 1 periodo:",
35       np.ptp(H(x, y)) / H(0.5, 0.3))
36 print("Fechamento da orbita |r(T) - r(0)|:",
37       np.hypot(x[-1] - x[0], y[-1] - y[0]))
38
39 L = 2.0
40 fig, ax = plt.subplots(figsize=(7, 7))
41
42 xg, yg = np.meshgrid(np.linspace(-L, L, 300), np.linspace(-L, L, 300))
43 u, v = campo(xg, yg)
44 ax.streamplot(xg, yg, u, v, color="0.75", density=1.2,
45              linewidth=0.6, arrowsize=0.8)
46
47 niveis = [0.05, 0.2, 0.45, 0.8, 1.25, 1.8]
48 cs = ax.contour(xg, yg, H(xg, yg), levels=niveis,
49                colors="tab:orange", linewidths=1.5)
50
51 for h in niveis:
52     x0 = np.sqrt(h / 4)
53     dx, dy = campo(x0, 0.0)
54     n = np.hypot(dx, dy)
55     ax.annotate(" ", xy=(x0 + 0.08 * dx / n, 0.08 * dy / n), xytext=(x0, 0),
56               arrowprops=dict(arrowstyle="->", color="tab:orange", lw=1.5))
57
58 mu, P = np.linalg.eigh(np.array([[4.0, -1.0], [-1.0, 1.0]]))
59 s = np.array([-L, L]) * 1.5
60
61 ax.plot(0, 0, "o", color="k", ms=8, zorder=5, label="$x^*, y^*$")
62
63 ax.set_xlim(-L, L)
64 ax.set_ylim(-L, L)
65 ax.set_aspect("equal")
66 ax.axhline(0, color="k", lw=0.5)
67 ax.axvline(0, color="k", lw=0.5)
68 ax.set_xlabel("$x$")
69 ax.set_ylabel("$y$")
70 ax.set_title(r"Ex. 1b: $\dot{x}=-x+y, \dot{y}=-4x+y$")
71 ax.legend(loc="upper left", fontsize=8.5, framealpha=0.95)
72 plt.tight_layout()
73 plt.savefig(os.path.join(IMG_DIR, "retrato_fase_1b.png"), dpi=200)
74 plt.show()

```

Ex. 1c) – ex1c.py

```

1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy.integrate import solve_ivp
5
6  IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
7  os.makedirs(IMG_DIR, exist_ok=True)
8
9
10 def campo(x, y):
11     return x * (3 - x - 2 * y), y * (2 - x - y) # def. campo
12
13
14 def rhs(t, r):
15     return campo(r[0], r[1]) # lado direito
16
17
18 def jac(x, y):
19     return np.array([[3 - 2 * x - 2 * y, -2 * x],
20                     [-y, 2 - x - 2 * y]]) # jacobiana
21
22
23 def classifica(J):
24     tau, delta = np.trace(J), np.linalg.det(J)
25     disc = tau**2 - 4 * delta
26     if disc < 0:
27         return "sela"
28     if disc > 0:
29         return "estável" if tau < 0 else "instável"
30     if disc < 0:
31         return "espiral estável" if tau < 0 else "espiral instável" if tau > 0 else "centro"
32     return "degenerado" # classificações dos pto
33
34 pontos = {"(0,0)": (0.0, 0.0), "(0,2)": (0.0, 2.0),
35           "(3,0)": (3.0, 0.0), "(1,1)": (1.0, 1.0)}

```

```

36
37 autos = {}
38 for nome, (x0, y0) in pontos.items():
39     assert np.allclose(campo(x0, y0), 0), nome
40     J = jac(x0, y0)
41     lam, V = np.linalg.eig(J)
42     autos[nome] = (lam, V)
43     print(f"{nome}: J = {J.tolist()}, tr = {np.trace(J):.3f}, "
44           f"det = {np.linalg.det(J):.3f}, lambda = {np.round(lam, 5)}, "
45           f"-> {classifica(J)}") # classificação
46
47 lam_s, V_s = autos["(1,1)"]
48 i_est = np.argmin(lam_s.real)
49 i_ins = np.argmax(lam_s.real)
50 eps = 1e-4
51 variedades = []
52 for i, sentido, cor in [(i_est, -1, "tab:purple"),
53                        (i_ins, +1, "tab:red")]:
54     for sgn in (+1, -1):
55         r0 = np.array([1.0, 1.0]) + sgn * eps * V_s[:, i].real
56         sol = solve_ivp(rhs, (0, sentido * (12 if sentido < 0 else 40)), r0, max_step=0.01,
57                        rtol=1e-10, atol=1e-12)
58         mask = (sol.y[0] > -0.5) & (sol.y[0] < 4) & (sol.y[1] > -0.5) & (sol.y[1] < 3)
59         variedades.append((sol.y[0][mask], sol.y[1][mask], cor))
60
61 cores_pf = {"sela": "tab:red", "estável": "k", "instável": "white"}
62
63 xmin, xmax, ymin, ymax = -0.3, 3.5, -0.3, 2.6
64 fig, ax = plt.subplots(figsize=(8, 6.2))
65 xg, yg = np.meshgrid(np.linspace(xmin, xmax, 400), np.linspace(ymin, ymax, 400))
66 u, v = campo(xg, yg)
67 ax.streamplot(xg, yg, u, v, color="0.72", density=1.4, linewidth=0.6, arrowsize=0.8)
68
69 s = np.linspace(xmin, xmax, 2)
70 ax.plot(s, (3 - s) / 2, "--", color="tab:green", lw=1.3)
71 ax.plot(s, 2 - s, "--", color="tab:orange", lw=1.3)
72 ax.axvline(0, color="tab:green", lw=1.3, ls="--")
73 ax.axhline(0, color="tab:orange", lw=1.3, ls="--")
74
75 for xs, ys, cor in variedades:
76     ax.plot(xs, ys, color=cor, lw=2.2)
77 # plots e anotações
78 for nome, (x0, y0) in pontos.items():
79     tipo = classifica(jac(x0, y0))
80     ax.plot(x0, y0, "o", ms=10, mfc=cores_pf[tipo], mec="k", mew=1.5, zorder=6)
81
82 ax.set_xlim(xmin, xmax)
83 ax.set_ylim(ymin, ymax)
84 ax.set_xlabel("$x$")
85 ax.set_ylabel("$y$")
86 ax.set_title(r"Ex. 1c: $\dot{x}=x(3-x-2y)$, $\dot{y}=y(2-x-y)$")
87 plt.tight_layout()
88 plt.savefig(os.path.join(IMG_DIR, "retrato_fase_1c_global.png"), dpi=200)
89
90 fig2, axs = plt.subplots(2, 2, figsize=(10, 10))
91 h = 0.35
92 for ax, (nome, (x0, y0)) in zip(axs.flat, pontos.items()):
93     J = jac(x0, y0)
94     lam, V = autos[nome]
95     xg, yg = np.meshgrid(np.linspace(x0 - h, x0 + h, 200), np.linspace(y0 - h, y0 + h, 200))
96     u, v = campo(xg, yg)
97     ax.streamplot(xg, yg, u, v, color="0.55", density=1.3, linewidth=0.7, arrowsize=0.9)
98     tipo_pf = classifica(J)
99     for k in range(2):
100         d = V[:, k].real / np.linalg.norm(V[:, k].real)
101         if tipo_pf == "sela":
102             cor, papel = ("tab:purple", "estável") if lam[k].real < 0 else ("tab:red", "instável")
103         else:
104             lenta = abs(lam[k].real) == min(abs(lam.real))
105             cor, papel = ("tab:blue", "lenta") if lenta else ("tab:orange", "rápida")
106         ax.plot([x0 - 1.2 * h * d[0], x0 + 1.2 * h * d[0]],
107               [y0 - 1.2 * h * d[1], y0 + 1.2 * h * d[1]], color=cor, lw=2,
108               label=r"$\lambda$={lam[k].real:.4f}$ ({papel}), $\mathbf{v}$={propto{d[0]:.3f},\{d[1]:.3f}}$")
109     tipo = tipo_pf
110     ax.plot(x0, y0, "o", ms=9, mfc=cores_pf[classifica(J)], mec="k", mew=1.5, zorder=6)
111     ax.set_xlim(x0 - h, x0 + h)
112     ax.set_ylim(y0 - h, y0 + h)
113     ax.set_aspect("equal")
114     ax.set_title(f"{nome}: {tipo}")
115     ax.set_xlabel("$x$")
116     ax.set_ylabel("$y$")
117 plt.suptitle("Ex. 1c: retratos de fase locais (campo não linear + autovetores de J)")
118 plt.tight_layout()
119 plt.savefig(os.path.join(IMG_DIR, "retrato_fase_1c_local.png"), dpi=200)
120 plt.show()

```

Ex. 2 – ex2.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.integrate import quad
5 from scipy.special import ellipk
6
7 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
8 os.makedirs(IMG_DIR, exist_ok=True)
9
10 s6 = np.sqrt(6.0)
11
12
13 def V1(th): # V_g (\theta)
14     return 1 - np.cos(th)
15
16
17 def V2(th): # V_f(\theta)

```

```

18         return th**2 / 2 - th**4 / 24
19
20 for th0 in (0.0, s6, -s6):
21     J = np.array([[0.0, 1.0], [-1 + th0**2 / 2, 0.0]])
22     print(f"theta* = {th0:+.5f}: autovalores = {np.round(np.linalg.eigvals(J), 6)}") # autovals
23
24
25 def T_pendolo(A):
26     return 4 * ellipk(np.sin(A / 2)**2) # periodo da eq. nao-aprox
27
28
29 def T_aprox(A):
30     f = lambda phi: 1 / np.sqrt(1 - A**2 * (1 + np.sin(phi)**2) / 12)
31     return 4 * quad(f, 0, np.pi / 2)[0] # T_aprox
32
33 #plots
34 fig, axs = plt.subplots(1, 3, figsize=(17, 5.4),
35                         gridspec_kw={"width_ratios": [1.35, 1, 0.9]})
36
37 ax = axs[0]
38 th = np.linspace(-2 * np.pi - 0.6, 2 * np.pi + 0.6, 600)
39 om = np.linspace(-3.2, 3.2, 500)
40 TH, OM = np.meshgrid(th, om)
41 E1 = OM**2 / 2 + V1(TH)
42 ax.contour(TH, OM, E1, levels=[0.1, 0.4, 0.9, 1.5, 2.6, 3.5, 4.5],
43            colors="tab:blue", linewidths=1)
44 ax.streamplot(TH, OM, OM, -np.sin(TH), color="0.8", density=0.9,
45              linewidth=0.5, arrowsize=0.7)
46 for n in range(-2, 3):
47     ax.plot(n * np.pi, 0, "o", ms=8, mec="k",
48           mfc="k" if n % 2 == 0 else "tab:red", zorder=5)
49 ax.set_xticks(np.arange(-2, 3) * np.pi)
50 ax.set_xticklabels([r"$-2\pi$", r"$-\pi$", "0", r"$\pi$", r"$2\pi$"])
51 ax.set_xlim(th[0], th[-1])
52 ax.set_title(r"(1) $\ddot{\theta}=-\sin\theta$")
53 ax.set_xlabel(r"$\theta$")
54 ax.set_ylabel(r"$\omega=\dot{\theta}$")
55
56 ax = axs[1]
57 th = np.linspace(-4.2, 4.2, 600)
58 om = np.linspace(-3.2, 3.2, 500)
59 TH, OM = np.meshgrid(th, om)
60 E2 = OM**2 / 2 + V2(TH)
61 dentro = np.abs(TH) < s6
62 ax.contour(TH, OM, np.where(dentro, E2, np.nan), levels=[0.1, 0.4, 0.8, 1.2],
63            colors="tab:blue", linewidths=1)
64 ax.contour(TH, OM, E2, levels=[1.8, 2.6, 3.6], colors="tab:green", linewidths=1)
65 ax.contour(TH, OM, np.where(dentro, np.nan, E2), levels=[-1.5, 0.0, 0.8],
66            colors="tab:purple", linewidths=1, linestyle="solid")
67 ax.streamplot(TH, OM, OM, -TH + TH**3 / 6, color="0.8", density=0.9,
68              linewidth=0.5, arrowsize=0.7)
69 ax.plot(0, 0, "o", ms=8, mfc="k", mec="k", zorder=5)
70 for th0 in (s6, -s6):
71     ax.plot(th0, 0, "o", ms=8, mfc="tab:red", mec="k", zorder=5)
72 ax.set_xlim(th[0], th[-1])
73 ax.set_ylim(om[0], om[-1])
74 ax.set_title(r"(2) $\ddot{\theta}=-\theta+\theta^3/6$")
75 ax.set_xlabel(r"$\theta$")
76 ax.set_ylabel(r"$\omega=\dot{\theta}$")
77
78 ax = axs[2]
79 A1 = np.linspace(0.01, np.pi - 1e-3, 400)
80 A2 = np.linspace(0.01, s6 - 1e-4, 400)
81 ax.plot(A1, [T_pendolo(a) for a in A1], color="tab:blue", label=r"(1) exato, $\pi$")
82 ax.plot(A2, [T_aprox(a) for a in A2], color="tab:orange", label=r"(2) aprox., $\sqrt{6}$")
83 ax.axhline(2 * np.pi, color="k", lw=0.8, ls=":", label=r"$2\pi$ (linear)")
84 ax.axvline(np.pi, color="tab:blue", lw=0.8, ls="--")
85 ax.axvline(s6, color="tab:orange", lw=0.8, ls="--")
86 ax.set_ylim(6, 20)
87 ax.set_xlabel("amplitude $A$")
88 ax.set_ylabel("periodo $T$")
89 ax.set_title("Período das oscilações")
90 ax.legend(fontsize=8.5, loc="upper left")
91
92 plt.tight_layout()
93 plt.savefig(os.path.join(IMG_DIR, "diagrama_fases_ex2.png"), dpi=200)
94
95 print("\n A      T_exato      T_aprox")
96 for A in (0.2, 0.5, 1.0, 1.5, 2.0, 2.4):
97     print(f"{A:4.1f}   {T_pendolo(A):8.5f}   {T_aprox(A):8.5f}")
98 plt.show()

```

Ex. 3 e 3a) – ex3a.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
6 os.makedirs(IMG_DIR, exist_ok=True)
7
8 NOME_PAR = "b"
9
10
11 def f(x, p): # mapa grau 2
12     return 1 - p * x**2
13
14
15 def df(x, p): # f'(x)
16     return -2 * p * x
17
18
19 def lyapunov(p, x0=0.1, n_trans=1000, n_med=5000, p_max=128, tol=1e-9): #calc lyap, contador de periodos
20     p = np.atleast_1d(np.asarray(p, dtype=float))
21     x = np.full_like(p, x0)
22     for _ in range(n_trans):

```

```

23     x = f(x, p)
24
25     orb = [x.copy()]
26     for _ in range(p_max):
27         orb.append(f(orb[-1], p))
28     orb = np.array(orb)
29     q = np.zeros(p.size, dtype=int)
30     for qq in range(1, p_max + 1):
31         ok = (q == 0) & np.all(np.abs(orb[qq:] - orb[:-qq]) < tol, axis=0)
32         q[ok] = qq
33     n_uso = np.where(q > 0, (n_med // np.maximum(q, 1)) * q, n_med)
34     soma = np.zeros_like(p)
35     for k in range(n_med):
36         soma += np.where(k < n_uso, np.log(np.abs(df(x, p)) + 1e-300), 0.0)
37     x = f(x, p)
38     return soma / n_uso
39
40
41 def orbita(p, x0, n): # def. orbita
42     x = np.empty(n + 1)
43     x[0] = x0
44     for k in range(n):
45         x[k + 1] = f(x[k], p)
46     return x
47
48 P_PER = 1.10 # periodico
49 P_CAO = 1.90 # caos
50 X0 = 0.10 # def. x0
51
52 lam_per = lyapunov(P_PER)[0] # lyap periodico
53 lam_cao = lyapunov(P_CAO)[0] # lyap caotico
54 print(f"lambda(b={P_PER}) = {lam_per:.4f}")
55 print(f"lambda(b={P_CAO}) = {lam_cao:.4f}")
56
57 P = np.linspace(1e-3, 2.0, 3000) # mapa de bifurcacao
58 x = np.full_like(P, X0)
59 for _ in range(1000):
60     x = f(x, P)
61 n_pts = 200
62 pts = np.empty((n_pts, P.size))
63 for k in range(n_pts):
64     x = f(x, P)
65     pts[k] = x
66 LAM = lyapunov(P) # lyap(b)
67
68 fig, (a1, a2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True,
69                             gridspec_kw={"height_ratios": [1.6, 1]})
70 a1.plot(np.tile(P, n_pts), pts.ravel(), "k", color="k", alpha=0.25)
71 a1.set_ylabel("$x_n$")
72 a1.set_title(r"Mapa de Grau 2: $x_{n+1}=1-b\backslash x_n^2$")
73 a2.plot(P, LAM, color="tab:blue", lw=0.7)
74 a2.axhline(0, color="k", lw=0.8)
75 a2.set_ylim(-3, 1)
76 a2.set_ylabel(r"$\lambda$")
77 a2.set_xlabel("$b$")
78 for p, cor in ((P_PER, "tab:green"), (P_CAO, "tab:red")):
79     for a in (a1, a2):
80         a.axvline(p, color=cor, lw=1, ls="--")
81 a2.text(P_PER, 0.6, f"$b={P_PER}$", color="tab:green")
82 a2.text(P_CAO, 0.6, f"$b={P_CAO}$", color="tab:red", ha="right")
83 plt.tight_layout()
84 plt.savefig(os.path.join(IMG_DIR, "ex3_bifurcacao_lyapunov.png"), dpi=200)
85
86 N = 80
87 xp = orbita(P_PER, X0, N) # serie periodica
88 xc = orbita(P_CAO, X0, N) # serie caotica
89
90 d2 = np.abs(xp[2:] - xp[:-2])
91 n_trans = next(n for n in range(d2.size) if np.all(d2[n:] < 1e-4)) # fim do transiente
92
93 fig, axs = plt.subplots(2, 1, figsize=(10, 6.5), sharex=True)
94 ax = axs[0]
95 ax.axvspan(0, n_trans, color="0.88", label=f"transiente ({n}<{n_trans})")
96 ax.plot(xp, "o-", ms=3, lw=0.8, color="tab:green",
97         label=r"$b={P_PER}$, $\lambda={lam_per:.3f}$ (período 2)")
98 ax.set_ylabel("$x_n$")
99 ax.legend(loc="lower right", fontsize=9)
100 ax = axs[1]
101 ax.plot(xc, "o-", ms=3, lw=0.8, color="tab:red",
102         label=r"$b={P_CAO}$, $\lambda={lam_cao:.3f}$ (caótico)")
103 ax.set_ylabel("$x_n$")
104 ax.set_xlabel("$n$")
105 ax.set_ylim(-1.35, 1.1)
106 ax.legend(loc="lower right", fontsize=9)
107 axs[0].set_title(r"Séries temporais, $x_0={X0}$")
108 plt.tight_layout()
109 plt.savefig(os.path.join(IMG_DIR, "ex3a_i_series.png"), dpi=200)
110
111 d0 = 1e-10 # sensibilidade as cond. iniciais
112 N2 = 60
113 fig, axs = plt.subplots(2, 2, figsize=(12, 7))
114 for col, (p, lam, cor) in enumerate(((P_PER, lam_per, "tab:green"),
115                                     (P_CAO, lam_cao, "tab:red"))):
116     x1 = orbita(p, X0, N2)
117     x2 = orbita(p, X0 + d0, N2)
118     n = np.arange(N2 + 1)
119     ax = axs[0, col]
120     ax.plot(n, x1, "o-", ms=3, lw=0.8, color=cor, label=r"$x_0={X0}$")
121     ax.plot(n, x2, "s--", ms=3, lw=0.8, color="k", mfc="none",
122            label=r"$x_0={X0}+10^{-{10}}$")
123     ax.set_title(r"$b={p}$, $\lambda={lam:.3f}$")
124     ax.set_ylabel("$x_n$")
125     ax.legend(fontsize=8, loc="lower left")
126     ax = axs[1, col]
127     ax.semilogy(n, np.abs(x2 - x1) + 1e-17, "o-", ms=3, lw=0.8, color=cor,
128                label=r"$|\delta_n|$")
129     ax.semilogy(n, d0 * np.exp(lam * n), "k:", lw=1.5,

```

```

130         label=r"$\delta_0\backslash e^{\lambda n}$")
131     ax.set_ylim(1e-17, 10)
132     ax.set_xlabel("$n$")
133     ax.set_ylabel(r"$\delta_n=|x_n-x_{n-1}|$")
134     ax.legend(fontsize=8, loc="lower right" if lam > 0 else "upper right")
135 plt.tight_layout()
136 plt.savefig(os.path.join(IMG_DIR, "ex3a_ii_sensibilidade.png"), dpi=200)
137
138
139 def cobweb(ax, p, x0, n_it, lam, cor): # cobweb
140     xs = np.linspace(-1, 1, 500)
141     ax.plot(xs, f(xs, p), color="k", lw=1.2, label=r"$f(x)=1-\{p\}, x^2$")
142     ax.plot(xs, xs, color="0.5", lw=0.8, ls="--", label="$x_{n+1}=x_n$")
143     x = x0
144     px, py = [x], [0.0]
145     for _ in range(n_it):
146         y = f(x, p)
147         px += [x, y]
148         py += [y, y]
149         x = y
150     ax.plot(px, py, color=cor, lw=1)
151     ax.plot(x0, 0, "o", color=cor, ms=5)
152     ax.set_xlim(-1, 1)
153     ax.set_ylim(-1, 1)
154     ax.set_aspect("equal")
155     ax.set_xlabel("$x_n$")
156     ax.set_ylabel("$x_{n+1}$")
157     ax.set_title(r"$b=\{p\}$, $\lambda=\{lam:.3f\}$, $x_0=\{x0\}$, $\{n\_it\}$ iterações")
158     ax.legend(fontsize=8, loc="lower center")
159
160 fig, axs = plt.subplots(1, 2, figsize=(12, 6))
161
162 x0_per = round(orbita(P_PER, X0, 200)[-1] + 0.05, 3) # x0 prox. do atrator
163 cobweb(axs[0], P_PER, x0_per, 10, lam_per, "tab:green")
164 cobweb(axs[1], P_CAO, X0, 10, lam_cao, "tab:red")
165 plt.tight_layout()
166 plt.savefig(os.path.join(IMG_DIR, "ex3a_iii_cobweb.png"), dpi=200)
167 plt.show()

```

Ex. 3b) – ex3b.py

```

1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy.optimize import brentq
5
6  IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
7  os.makedirs(IMG_DIR, exist_ok=True)
8
9
10 def f(x, p): # mapa grau 2
11     return 1 - p * x**2
12
13
14 def df(x, p): # f'(x)
15     return -2 * p * x
16
17
18 def lyapunov(p, x0=0.1, n_trans=2000, n_med=5000, p_max=128, tol=1e-9): # calc lyap, contador de periodos
19     p = np.atleast_1d(np.asarray(p, dtype=float))
20     x = np.full_like(p, x0)
21     for _ in range(n_trans):
22         x = f(x, p)
23
24     orb = [x.copy()]
25     for _ in range(p_max):
26         orb.append(f(orb[-1], p))
27     orb = np.array(orb)
28     q = np.zeros(p.size, dtype=int)
29     for qq in range(1, p_max + 1):
30         ok = (q == 0) & np.all(np.abs(orb[qq:] - orb[:-qq]) < tol, axis=0)
31         q[ok] = qq
32     n_uso = np.where(q > 0, (n_med // np.maximum(q, 1)) * q, n_med)
33     soma = np.zeros_like(p)
34     for k in range(n_med):
35         soma += np.where(k < n_uso, np.log(np.abs(df(x, p)) + 1e-300), 0.0)
36         x = f(x, p)
37     return soma / n_uso
38
39
40 def orbita(p, x0, n): # def. orbita
41     x = np.empty(n + 1)
42     x[0] = x0
43     for k in range(n):
44         x[k + 1] = f(x[k], p)
45     return x
46
47
48 def ciclo(b, x, per, tol=1e-14, itmax=100): # newton em f~p(x) - x, retorna \mu
49     for _ in range(itmax):
50         y, mu = x, 1.0
51         for _ in range(per):
52             mu *= df(y, b)
53             y = f(y, b)
54         dx = (y - x) / (mu - 1)
55         x -= dx
56         if abs(dx) < tol:
57             break
58     y, mu = x, 1.0
59     for _ in range(per):
60         mu *= df(y, b)
61         y = f(y, b)
62     return x, mu
63
64

```

```

65 def mu_de(b, x_guess, per): # multiplicador ciclo
66     return ciclo(b, x_guess, per)[1]
67
68 N_BIF = 6
69 b_bif = [0.75]
70 x_ref = 0.0
71 for k in range(1, N_BIF): # duplicacoes \rightarrow mu = -1
72     per = 2**k
73
74     passo = (b_bif[-1] - b_bif[-2]) / 4.67 if k > 1 else 0.25
75     b = b_bif[-1] + 0.3 * passo
76     x = orbita(b, 0.1, 200000)[-1]
77     x, mu = ciclo(b, x, per)
78
79     h = passo / 50
80     while True:
81         b_novo = b + h
82         x_novo, mu_novo = ciclo(b_novo, x, per)
83         if mu_novo < -1:
84             break
85         b, x, mu = b_novo, x_novo, mu_novo
86     b_n = brentq(lambda bb: mu_de(bb, x, per) + 1, b, b_novo, xtol=1e-15)
87     b_bif.append(b_n)
88
89 print("Duplicacoes de periodo (mu = -1):")
90 for k, b_n in enumerate(b_bif):
91     per = 2**k
92     x_c = orbita(b_n, 0.1, 100000)[-1]
93     if per > 1:
94         x_c, mu = ciclo(b_n, x_c, per)
95     else:
96         x_c = (-1 + np.sqrt(1 + 4 * b_n)) / (2 * b_n)
97         mu = df(x_c, b_n)
98     lam_ciclo = np.log(abs(mu)) / per
99     lam_medio = lyapunov(b_n, n_trans=20000, n_med=200000)[0]
100    print(f" {per:3d} -> {2*per:3d}: b = {b_n:.10f}, mu = {mu:+.3e}, "
101          f"lambda_ciclo = {lam_ciclo:+.2e}, lambda_medio = {lam_medio:+.2e}")
102
103 B0, B1 = 0.5, 1.45
104 P = np.linspace(B0, B1, 3000) # mapa de bifurcacao
105 x = np.full_like(P, 0.1)
106 for _ in range(2000):
107     x = f(x, P)
108 n_pts = 256
109 pts = np.empty((n_pts, P.size))
110 for k in range(n_pts):
111     x = f(x, P)
112     pts[k] = x
113 LAM = lyapunov(P)
114
115 fig, (a1, a2) = plt.subplots(2, 1, figsize=(11, 8), sharex=True,
116                             gridspec_kw={"height_ratios": [1.6, 1]})
117 a1.plot(np.tile(P, n_pts), pts.ravel(), "k", alpha=0.3)
118 a1.set_ylabel("$x$")
119 a1.set_title(r"Rota para o caos por duplicação de período: $x_{n+1}=1-b\backslash x_n^2$")
120 a2.plot(P, LAM, color="tab:blue", lw=0.8)
121 a2.axhline(0, color="k", lw=0.8)
122 a2.set_ylim(-1.5, 0.4)
123 a2.set_ylabel(r"$\lambda$")
124 a2.set_xlabel("$b$")
125 for k, b_n in enumerate(b_bif[:4]):
126     for a in (a1, a2):
127         a.axvline(b_n, color="tab:red", lw=0.8, ls="--")
128         a1.text(b_n, 1.04, rf"$b_{k+1}$", color="tab:red", ha="center", fontsize=9)
129 a1.set_ylim(-0.5, 1.1)
130 plt.tight_layout()
131 plt.savefig(os.path.join(IMG_DIR, "ex3b_rota_duplicacao.png"), dpi=200)
132
133 fig, axs = plt.subplots(1, 4, figsize=(16, 4.2)) # lyap qse 0 nas duplicacoes
134 for k, ax in enumerate(axs):
135     b_n = b_bif[k]
136     larg = 0.02 * 4.67**(-k) * 4
137     bb = np.linspace(b_n - larg, b_n + larg, 401)
138     ll = lyapunov(bb, n_trans=20000, n_med=20000)
139     ax.plot(bb, ll, color="tab:blue", lw=1)
140     ax.axhline(0, color="k", lw=0.8)
141     ax.axvline(b_n, color="tab:red", lw=0.8, ls="--")
142     ax.plot(b_n, 0, "o", color="tab:red", ms=6)
143     ax.set_title(rf"$\{2**k\}\to\{2**(k+1)\}$: $b_{k+1}=\{b_n:.6f\}$", fontsize=10)
144     ax.set_xlabel("$b$")
145     ax.ticklabel_format(axis="x", style="plain", useOffset=False)
146     ax.tick_params(axis="x", labelrotation=30)
147     axs[0].set_ylabel(r"$\lambda$")
148     plt.suptitle(r"3b-i: $\lambda$ em cada duplicação de período")
149     plt.tight_layout()
150     plt.savefig(os.path.join(IMG_DIR, "ex3b_i_zoom_lyapunov.png"), dpi=200)
151
152 casos = [(0.60, "período 1"), (1.10, "período 2"), (1.32, "período 4"), # series temporais
153          (1.385, "período 8"), (1.397, "período 16"), (1.45, "caótico")]
154 N_TR, N_SH = 50000, 48
155 fig, axs = plt.subplots(len(casos), 1, figsize=(10, 11), sharex=True)
156 for ax, (b, rot) in zip(axs, casos):
157     xs = orbita(b, 0.1, N_TR + N_SH)[N_TR:]
158     lam = lyapunov(b, n_trans=20000, n_med=50000)[0]
159     ax.plot(np.arange(N_SH + 1), xs, "o-", ms=3, lw=0.8,
160            label=rf"$b=\{b\}$, $\lambda=\{lam:+.3f\}$ ({rot})")
161     ax.set_ylabel("$x_n$")
162     ax.set_ylim(-1.1, 1.15)
163     ax.legend(loc="lower right", fontsize=8.5, framealpha=0.95)
164     axs[-1].set_xlabel(rf"$n$ (após descartar {N_TR} iterações de transiente)")
165     axs[0].set_title("Séries temporais ao longo da cascata de duplicações")
166     plt.tight_layout()
167     plt.savefig(os.path.join(IMG_DIR, "ex3b_series.png"), dpi=200)
168     plt.show()

```

Ex. 3c)l) – ex3ci.py

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.optimize import brentq
5
6 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
7 os.makedirs(IMG_DIR, exist_ok=True)
8
9
10 def f(x, b): # mapa grau 2
11     return 1 - b * x**2
12
13
14 def df(x, b): # f'(x)
15     return -2 * b * x
16
17
18 def f_n(x, b, n): # f^n(x)
19     for _ in range(n):
20         x = f(x, b)
21     return x
22
23
24 def lyapunov(b, x0=0.1, n_trans=5000, n_med=20000, p_max=128, tol=1e-9): # calc lyap, contador de periodos
25     b = np.atleast_1d(np.asarray(b, dtype=float))
26     x = np.full_like(b, x0)
27     for _ in range(n_trans):
28         x = f(x, b)
29
30     orb = [x.copy()]
31     for _ in range(p_max):
32         orb.append(f(orb[-1], b))
33     orb = np.array(orb)
34     q = np.zeros(b.size, dtype=int)
35     for qq in range(1, p_max + 1):
36         ok = (q == 0) & np.all(np.abs(orb[qq:] - orb[:-qq]) < tol, axis=0)
37         q[ok] = qq
38     n_uso = np.where(q > 0, (n_med // np.maximum(q, 1)) * q, n_med)
39     soma = np.zeros_like(b)
40     for k in range(n_med):
41         soma += np.where(k < n_uso, np.log(np.abs(df(x, b)) + 1e-300), 0.0)
42         x = f(x, b)
43     return soma / n_uso
44
45
46 def orbita(b, x0, n): # def. orbita
47     x = np.empty(n + 1)
48     x[0] = x0
49     for k in range(n):
50         x[k + 1] = f(x[k], b)
51     return x
52
53
54 def ciclo(b, x, per, tol=1e-14, itmax=200): # newton em f^n(x) - x, retorna \mu
55     for _ in range(itmax):
56         y, mu = x, 1.0
57         for _ in range(per):
58             mu *= df(y, b)
59             y = f(y, b)
60         dx = (y - x) / (mu - 1)
61         x -= dx
62         if abs(dx) < tol:
63             break
64     y, mu, pts = x, 1.0, []
65     for _ in range(per):
66         pts.append(y)
67         mu *= df(y, b)
68         y = f(y, b)
69     return np.array(pts), mu
70
71
72 def periodo_atrator(B, n_trans=20000, p_max=64, tol=1e-6): # contador de periodos
73     x = np.full_like(B, 0.1)
74     for _ in range(n_trans):
75         x = f(x, B)
76     orb = [x.copy()]
77     for _ in range(p_max):
78         x = f(x, B)
79         orb.append(x.copy())
80     orb = np.array(orb)
81     per = np.zeros(B.size, dtype=int)
82     for p in range(1, p_max + 1):
83         ok = (per == 0) & np.all(np.abs(orb[p:] - orb[:-p]) < tol, axis=0)
84         per[ok] = p
85     return per
86
87 B_scan = np.linspace(1e-3, 2.0, 20000) # varredura do periodo
88 PER = periodo_atrator(B_scan)
89 m3 = PER == 3
90 print(f"varredura 0 < b <= 2: atrator de periodo 3 apenas em "
91       f"{B_scan[m3].min():.5f} <= b <= {B_scan[m3].max():.5f} ({m3.sum()} valores de b)")
92
93 B_T = 7 / 4
94 b_in = 1.76
95 x_in = orbita(b_in, 0.1, 20000)[-1]
96 pts_in, mu_in = ciclo(b_in, x_in, 3)
97
98
99 def mu3(b): # multiplicador ciclo de periodo 3
100     return ciclo(b, x_in, 3)[1]
101
102 b_sup = brentq(lambda b: f_n(0.0, b, 3), 1.751, 1.76, xtol=1e-15)
103
104 b_dup = brentq(lambda b: mu3(b) + 1, b_sup, 1.775, xtol=1e-15) # fim da janela: \mu_3 = -1
105
```

```

106 print(f"inicio da janela          = {B_T:.10f}")
107 print(f"fim do periodo 3 estavel = {b_dup:.10f}")
108 print(f"b = {b_in}: ciclo = {np.round(np.sort(pts_in), 6)}, mu3 = {mu_in:+.6f}, "
109       f"lambda_ciclo = {np.log(abs(mu_in))/3:+.6f}, "
110       f"lambda_media = {lyapunov(b_in, n_trans=20000, n_med=200000)[0]:+.6f}")
111
112 B0, B1 = 1.70, 1.80
113 P = np.linspace(B0, B1, 3000) # zoom na janela
114 x = np.full_like(P, 0.1)
115 for _ in range(3000):
116     x = f(x, P)
117 n_pts = 300
118 pts = np.empty((n_pts, P.size))
119 for k in range(n_pts):
120     x = f(x, P)
121     pts[k] = x
122 LAM = lyapunov(P)
123
124 fig, (a1, a2) = plt.subplots(2, 1, figsize=(11, 8), sharex=True,
125                             gridspec_kw={"height_ratios": [1.6, 1]})
126 a1.plot(np.tile(P, n_pts), pts.ravel(), "k", color="k", alpha=0.3)
127 a1.set_ylabel("$x$")
128 a1.set_title(r"Janela de periodo 3: $x_{n+1}=1-b\backslash x_n^2$")
129 a2.plot(P, LAM, color="tab:blue", lw=0.8)
130 a2.axhline(0, color="k", lw=0.8)
131 a2.set_ylim(-1.5, 0.8)
132 a2.set_ylabel(r"$\lambda$")
133 a2.set_xlabel("$b$")
134 for bb, rot, cor in ((B_T, r"$b=7/4$", "tab:red"),
135                     (b_dup, r"$b_{3\to6}$", "tab:purple")):
136     for a in (a1, a2):
137         a.axvline(bb, color=cor, lw=0.9, ls="--")
138         a1.text(bb, 1.04, rot, color=cor, ha="center", fontsize=9)
139 a1.axvspan(B_T, b_dup, color="tab:green", alpha=0.08)
140 a2.axvspan(B_T, b_dup, color="tab:green", alpha=0.08)
141 for bb, cor in ((1.745, "tab:orange"), (1.76, "tab:green")):
142     a2.plot(bb, lyapunov(bb)[0], "o", color=cor, ms=6)
143 a1.set_ylim(-1.05, 1.1)
144 plt.tight_layout()
145 plt.savefig(os.path.join(IMG_DIR, "ex3c_i_janela.png"), dpi=200)
146
147 N_TR, N_SH = 20000, 90
148 fig, axs = plt.subplots(2, 1, figsize=(11, 6.5), sharex=True) # series temporais
149 for ax, (b, cor, rot) in zip(axs, ((1.745, "tab:orange", "caótico"),
150                                   (1.76, "tab:green", "periodo 3"))):
151     xs = orbita(b, 0.1, N_TR + N_SH)[N_TR:]
152     lam = lyapunov(b, n_trans=20000, n_med=200000)[0]
153     ax.plot(np.arange(N_SH + 1), xs, "o-", ms=3, lw=0.8, color=cor,
154            label=r"$b={b}$, $\lambda={lam:+.3f}$ ({rot})")
155     ax.set_ylabel("$x_n$")
156     ax.set_ylim(-1.45, 1.1)
157     ax.legend(loc="lower right", fontsize=9, framealpha=0.95)
158 axs[1].set_xlabel(rf"$n$ (após descartar {N_TR} iterações de transiente)")
159 axs[0].set_title("Séries temporais: região caótica e janela de periodo 3")
160 plt.tight_layout()
161 plt.savefig(os.path.join(IMG_DIR, "ex3c_i_series.png"), dpi=200)
162
163 plt.show()

```

Ex. 3c)ii) – ex3cii.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.optimize import brentq
5
6 IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imagens")
7 os.makedirs(IMG_DIR, exist_ok=True)
8
9
10 def f(x, b):
11     return 1 - b * x**2
12
13
14 def df(x, b):
15     return -2 * b * x
16
17
18 def lyapunov(b, x0=0.1, n_trans=20000, n_med=20000, p_max=128, tol=1e-9):
19     b = np.atleast_1d(np.asarray(b, dtype=float))
20     x = np.full_like(b, x0)
21     for _ in range(n_trans):
22         x = f(x, b)
23
24     orb = [x.copy()]
25     for _ in range(p_max):
26         orb.append(f(orb[-1], b))
27     orb = np.array(orb)
28     q = np.zeros(b.size, dtype=int)
29     for qq in range(1, p_max + 1):
30         ok = (q == 0) & np.all(np.abs(orb[qq:] - orb[:-qq]) < tol, axis=0)
31         q[ok] = qq
32     n_uso = np.where(q > 0, (n_med // np.maximum(q, 1)) * q, n_med)
33     soma = np.zeros_like(b)
34     for k in range(n_med):
35         soma += np.where(k < n_uso, np.log(np.abs(df(x, b)) + 1e-300), 0.0)
36         x = f(x, b)
37     return soma / n_uso
38
39
40 def orbita(b, x0, n):
41     x = np.empty(n + 1)
42     x[0] = x0
43     for k in range(n):
44         x[k + 1] = f(x[k], b)

```

```

45     return x
46
47
48 def ciclo(b, x, per, tol=1e-14, itmax=200):
49     for _ in range(itmax):
50         y, mu = x, 1.0
51         for _ in range(per):
52             mu *= df(y, b)
53             y = f(y, b)
54             dx = (y - x) / (mu - 1)
55             x -= dx
56             if abs(dx) < tol:
57                 break
58     y, mu = x, 1.0
59     for _ in range(per):
60         mu *= df(y, b)
61         y = f(y, b)
62     return x, mu
63
64
65 def periodo_atrator(B, n_trans=100000, p_max=96, tol=1e-7):
66     x = np.full_like(B, 0.1)
67     for _ in range(n_trans):
68         x = f(x, B)
69     orb = [x.copy()]
70     for _ in range(p_max):
71         x = f(x, B)
72         orb.append(x.copy())
73     orb = np.array(orb)
74     per = np.zeros(B.size, dtype=int)
75     for p in range(1, p_max + 1):
76         ok = (per == 0) & np.all(np.abs(orb[p:] - orb[:-p]) < tol, axis=0)
77         per[ok] = p
78     return per
79
80 B_scan = np.linspace(1.765, 1.790, 10000)
81 PER = periodo_atrator(B_scan)
82 print("Varredura do periodo do atrator (1.765 <= b <= 1.790):")
83 for p in (3, 6, 12, 24, 48):
84     idx = np.flatnonzero(PER == p)
85     if idx.size:
86
87         fim = idx[0]
88         while fim + 1 < PER.size and PER[fim + 1] == p:
89             fim += 1
90         print(f" periodo {p:3d}: {B_scan[idx[0]]:.5f} <= b <= {B_scan[fim]:.5f}")
91
92 N_DUP = 5
93 b_dup = []
94 b = 1.760
95 x = orbita(b, 0.1, 100000)[-1]
96 for k in range(N_DUP):
97     per = 3 * 2**k
98     x, mu = ciclo(b, x, per)
99     h = 1e-4 if k == 0 else (b_dup[-1] - (b_dup[-2] if k > 1 else 1.75)) / 200
100    while True:
101        x_n, mu_n = ciclo(b + h, x, per)
102        if mu_n < -1:
103            break
104        b, x, mu = b + h, x_n, mu_n
105    b_k = brentq(lambda bb: ciclo(bb, x, per)[1] + 1, b, b + h, xtol=1e-15)
106    b_dup.append(b_k)
107
108    b = b_k + 0.3 * h
109    x = orbita(b, x, 200000)[-1]
110
111    print("\nDuplicacoes dentro da janela (mu = -1 => lambda = 0):")
112    for k, b_k in enumerate(b_dup):
113        per = 3 * 2**k
114        print(f" {per:3d} -> {2*per:3d}: b = {b_k:.8f}")
115    b_inf_est = b_dup[-1] + (b_dup[-1] - b_dup[-2]) / (4.669 - 1)
116    print(f" acumulacao estimada: b - {b_inf_est:.5f}")
117
118    B0, B1 = 1.765, 1.790
119    P = np.linspace(B0, B1, 3000)
120    x = np.full_like(P, 0.1)
121    for _ in range(20000):
122        x = f(x, P)
123    n_pts = 384
124    pts = np.empty((n_pts, P.size))
125    for k in range(n_pts):
126        x = f(x, P)
127        pts[k] = x
128    LAM = lyapunov(P)
129
130    fig, (a1, a2) = plt.subplots(2, 1, figsize=(11, 8), sharex=True,
131                                gridspec_kw={"height_ratios": [1.6, 1]})
132    a1.plot(np.tile(P, n_pts), pts.ravel(), ",", color="k", alpha=0.35)
133    a1.set_ylabel("$x$")
134    a1.set_title(r"Duplicações de periodo dentro da janela de periodo 3: $x_{n+1}=1-b\cdot x_n^2$")
135    a2.plot(P, LAM, color="tab:blue", lw=0.8)
136    a2.axhline(0, color="k", lw=0.8)
137    a2.set_ylim(-1.0, 0.5)
138    a2.set_ylabel(r"$\lambda$")
139    a2.set_xlabel(r"$b$")
140    rot = [r"$3\to6$", r"$6\to12$", r"$12\to24$", r"$24\to48$"]
141    for k in range(4):
142        for a in (a1, a2):
143            a.axvline(b_dup[k], color="tab:red", lw=0.8, ls="--")
144            a1.text(b_dup[k], 1.05, rot[k], color="tab:red", ha="center" if k < 2 else ("right" if k == 2 else "left"), fontsize=8.5)
145    a1.set_ylim(-0.85, 1.1)
146    plt.tight_layout()
147    plt.savefig(os.path.join(IMG_DIR, "ex3c_ii_cascata.png"), dpi=200)
148
149
150 def ramo_central(B, p, n_trans=100000, n_pts=300):
151     x = np.full_like(B, 0.1)

```

```

152     for _ in range(n_trans):
153         x = f(x, B)
154     fases = [x.copy()]
155     for _ in range(p - 1):
156         x = f(x, B)
157         fases.append(x.copy())
158     fases = np.array(fases)
159     j0 = np.argmin(np.abs(fases), axis=0)
160     x = fases[j0, np.arange(B.size)]
161     out = np.empty((n_pts, B.size))
162     for k in range(n_pts):
163         for _ in range(p):
164             x = f(x, B)
165         out[k] = x
166     return out
167
168 rot_dup = [r"$3\t06$", r"$6\t012$", r"$12\t024$", r"$24\t048$", r"$48\t096$"]
169 b_ant = [1.75] + b_dup[:-1]
170 fig, axs = plt.subplots(2, 2, figsize=(13, 9))
171 for k, ax in enumerate(axs.flat):
172     esq = b_dup[k] - 0.3 * (b_dup[k] - b_ant[k])
173     dir_ = b_inf_est + 0.6 * (b_inf_est - b_dup[k])
174     PK = np.linspace(esq, dir_, 2500)
175     per = 3 * 2**k
176     pts_k = ramo_central(PK, per)
177     ax.plot(np.tile(PK, pts_k.shape[0]), pts_k.ravel(), ",", color="k", alpha=0.4)
178     lo, hi = np.percentile(pts_k, [0.2, 99.8])
179     pad = 0.08 * (hi - lo)
180     ax.set_ylim(lo - pad, hi + pad)
181     ax.set_xlim(esq, dir_)
182     for kk in range(k, len(b_dup)):
183         ax.vline(b_dup[kk], color="tab:red", lw=0.8, ls="--")
184         ax.vline(b_inf_est, color="tab:purple", lw=0.9, ls=":")
185         y_txt = hi + 0.5 * pad
186         for i_t, kk in enumerate(range(k, min(k + 3, len(b_dup)))):
187             ax.text(b_dup[kk], y_txt - i_t * 0.9 * pad, rot_dup[kk], color="tab:red", fontsize=8,
188                     ha="right", va="center")
189             ax.text(b_inf_est, lo - 0.5 * pad, r"$b_{\infty}$", color="tab:purple", fontsize=9,
190                     va="center")
191             ax.set_title(rf"zoom {k+1}: ramo $x\approx 0$ amostrado a cada {per} iterações", fontsize=10)
192             ax.set_xlabel("$b$")
193             ax.set_ylabel("$x$")
194             ax.ticklabel_format(axis="both", style="plain", useOffset=False)
195             ax.tick_params(axis="x", labelrotation=20)
196 plt.suptitle("Cascata dentro da janela de período 3: zooms sucessivos com eixos reescalados")
197 plt.tight_layout()
198 plt.savefig(os.path.join(IMG_DIR, "ex3c_ii_zoom.png"), dpi=200)
199
200 bb = np.linspace(b_inf_est + 1e-4, 1.790, 400)
201 ll = lyapunov(bb)
202 b_cao = round(float(bb[np.argmax(ll > 0.05)]), 4)
203 casos = [(1.7660, "período 3"),
204          (round((b_dup[0] + b_dup[1]) / 2, 4), "período 6"),
205          (round((b_dup[1] + b_dup[2]) / 2, 4), "período 12"),
206          (round((b_dup[2] + b_dup[3]) / 2, 5), "período 24"),
207          (b_cao, "caótico")]
208 N_TR, N_SH = 200000, 36
209 N_R = 48
210 fig, axs = plt.subplots(len(casos), 2, figsize=(14, 11),
211                         gridspec_kw={"width_ratios": [1, 1.25]})
212 for row, (b, r) in enumerate(casos):
213     xs = orbita(b, 0.1, N_TR + 3 * N_R + 3)[N_TR:]
214     per = periodo_atrator(np.array([b]))[0]
215     lam = lyapunov(b, n_trans=100000, n_med=200000)[0]
216     rot_leg = rf"$b={b}$, $\lambda={lam:+.3f}$ ({r})"
217
218     ax = axs[row, 0]
219     ax.plot(np.arange(N_SH + 1), xs[:N_SH + 1], "o-", ms=3, lw=0.7, color="tab:blue")
220     ax.set_ylim(-1.0, 1.1)
221     ax.set_ylabel("$x_n$")
222     ax.set_title(rot_leg, fontsize=9.5)
223
224     ax = axs[row, 1]
225     k0 = int(np.argmax(np.abs(xs[:3]) < 0.2))
226     sub = xs[k0::3][:N_R]
227     ax.plot(np.arange(sub.size), sub, "o-", ms=3.5, lw=0.7, color="tab:red")
228     ax.set_ylim(-0.1, 0.05)
229     ax.set_ylabel(r"$x_{\{3m\}}$ (ramo $x\approx 0$)")
230     ax.set_title(rot_leg, fontsize=9.5)
231     print(f"série b = {b}: período detectado = {per if per else 'nenhum <= 96'}, lambda = {lam:+.4f}")
232     axs[-1, 0].set_xlabel("$n$")
233     axs[-1, 1].set_xlabel(r"$m$ (retornos ao ramo $x\approx 0$, a cada 3 iterações)")
234     plt.suptitle(rf"Séries temporais ao longo da cascata (após descartar {N_TR} iterações de transiente)")
235     plt.tight_layout()
236     plt.savefig(os.path.join(IMG_DIR, "ex3c_ii_series.png"), dpi=200)
237
238 d0 = 1e-10
239 N_D = 600
240 casos_d = [(1.7660, "período 3"), (1.7782, "período 12"),
241            (1.77947, "período 24"), (1.7803, "caótico")]
242 fig, axs = plt.subplots(2, 2, figsize=(11, 7.5), sharey=True)
243 for ax, (b, r) in zip(axs.flat, casos_d):
244     lam = lyapunov(b, n_trans=100000, n_med=200000)[0]
245     xa = orbita(b, 0.1, 200000)[-1]
246     x1 = orbita(b, xa, N_D)
247     x2 = orbita(b, xa + d0, N_D)
248     n = np.arange(N_D + 1)
249     dd = np.abs(x2 - x1)
250     cor = "tab:red" if lam > 0 else "tab:green"
251
252     n_max = N_D if lam > 0 else int(min(N_D, 1.15 * np.argmax((dd < 1e-16) & (n > 0))))
253     ax.semilogy(n[:n_max + 1], dd[:n_max + 1] + 1e-18, "-", lw=0.9, color=cor,
254                label=r"$|\delta_n|$ (simulação)")
255     ax.semilogy(n[:n_max + 1], d0 * np.exp(lam * n[:n_max + 1]), "k--", lw=1.2,
256                label=r"$|\delta_0|e^{-\lambda n}$")
257     ax.axhline(d0, color="0.6", lw=0.6, ls=":")
258     ax.set_xlim(0, n_max)

```

```

259     ax.set_ylim(1e-17, 1)
260     ax.set_title(rf"$b = {b}$ ({r}), $\lambda = {lam:+.3f}$", fontsize=10.5)
261     ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.6)
262     ok = (dd > 1e-15) & (dd < 1e-4)
263     if lam > 0:
264         ok &= n < np.argmax(dd > 1e-4)
265     taxa = np.polyfit(n[ok], np.log(dd[ok]), 1)[0] if ok.sum() > 20 else np.nan
266     print(f"distanciamento b = {b}: lambda = {lam:+.4f}, taxa ajustada = {taxa:+.4f}")
267 for ax in axs[1]:
268     ax.set_xlabel("$n$")
269 for ax in axs[:, 0]:
270     ax.set_ylabel(r"$\delta_n = |x_n - x_n|$")
271 from matplotlib.lines import Line2D
272 h = [Line2D([], [], color="tab:green", lw=1.2), Line2D([], [], color="tab:red", lw=1.2),
273      Line2D([], [], color="k", ls="--", lw=1.2)]
274 l = [r"$\delta_n$, $\lambda < 0$ (periódico)", r"$\delta_n$, $\lambda > 0$ (caótico)",
275      r"$\delta_0$, $e^{-\lambda n}$"]
276 fig.legend(h, l, loc="upper center", ncol=3, frameon=False, bbox_to_anchor=(0.5, 0.945))
277 fig.suptitle(r"Distanciamento de órbitas vizinhas dentro da janela ($\delta_0 = 10^{-10}$)")
278 plt.tight_layout(rect=(0, 0, 1, 0.92))
279 plt.savefig(os.path.join(IMG_DIR, "ex3c_ii_distanciamento.png"), dpi=200)
280 plt.show()

```

Ex. 4 – ex4.py

```

1  import numpy as np
2  from scipy.optimize import brentq
3  import mpmath as mp
4
5  mp.mp.dps = 40
6
7  MAPAS = { # mapa grau 2
8            "Grau 2": dict(f=lambda x, a: 1 - a * x**2,
9                          df=lambda x, a: -2 * a * x,
10                         fm=lambda x, a: 1 - a * x**2,
11                         dfm=lambda x, a: -2 * a * x,
12                         a1=0.75, a_ini=0.60, nome_par="b"),
13          }
14  N_DUP = 7
15
16
17 def ciclo(m, a, x, p, tol=1e-14, itmax=200): # newton em f^p(x) - x, retorna \mu
18     f, df = m["f"], m["df"]
19     for _ in range(itmax):
20         y, mu = x, 1.0
21         for _ in range(p):
22             mu *= df(y, a)
23             y = f(y, a)
24         dx = (y - x) / (mu - 1)
25         x -= dx
26         if abs(dx) < tol:
27             break
28     y, mu = x, 1.0
29     for _ in range(p):
30         mu *= df(y, a)
31         y = f(y, a)
32     return x, mu
33
34
35 def ciclo_mp(m, a, x, p, itmax=40): # precisao
36     f, df = m["fm"], m["dfm"]
37     x = mp.mpf(x)
38     for _ in range(itmax):
39         y, mu = x, mp.mpf(1)
40         for _ in range(p):
41             mu *= df(y, a)
42             y = f(y, a)
43         dx = (y - x) / (mu - 1)
44         x -= dx
45         if abs(dx) < mp.mpf(10) ** (-32):
46             break
47     y, mu = x, mp.mpf(1)
48     for _ in range(p):
49         mu *= df(y, a)
50         y = f(y, a)
51     return x, mu
52
53
54 def orbita(m, a, x, n): # def. orbita
55     for _ in range(n):
56         x = m["f"](x, a)
57     return x
58
59
60 def periodo_minimo_ok(m, a, x, p): # evita ciclo de periodo p/2
61     if p == 1:
62         return True
63     y = orbita(m, a, x, p // 2)
64     return abs(y - x) > 1e-8
65
66
67 def duplicacoes(m): # a_n: \mu = -1
68     a_n = []
69     a = m["a_ini"]
70     h = 1e-3
71     x = orbita(m, a, 0.1, 100000)
72     for k in range(N_DUP):
73         p = 2**k
74         x, mu = ciclo(m, a, x, p)
75         assert periodo_minimo_ok(m, a, x, p), f"Newton convergiu para ciclo de periodo menor (p={p})"
76         while True:
77             x_novo, mu_novo = ciclo(m, a + h, x, p)
78             if mu_novo < -1:
79                 break
80         a, x, mu = a + h, x_novo, mu_novo

```

```

81     a_k = brentq(lambda aa: ciclo(m, aa, x, p)[1] + 1, a, a + h, xtol=1e-15, rtol=1e-15) # brent
82
83     g = lambda aa: ciclo_mp(m, aa, x, p)[1] + 1
84     a_mp = mp.findroot(g, (mp.mpf(a_k), mp.mpf(a_k) + mp.mpf("1e-12")), # refinamento
85                        solver="secant", tol=mp.mpf(10) ** (-28))
86     a_n.append(a_mp)
87
88     D_est = 0.1 if k == 0 else float(a_n[-1] - a_n[-2]) / 4.67
89     a = float(a_mp) + 0.3 * D_est
90     h = D_est / 200
91     x = orbita(m, a, x, 200000)
92     return a_n
93
94 for nome, m in MAPAS.items(): # tabela a_n, D_n, \delta_n
95     a_n = duplicacoes(m)
96     print(f"\n{nome}      ({m['nome_par']}_1 exato = {m['a1']:.10f})")
97     print(f"{'bifurcacao':>12} {'a_n':>16} {'D_n':>14} {'delta_n':>10}")
98     D = [None]
99     for k in range(1, len(a_n)):
100         D.append(a_n[k] - a_n[k - 1])
101     for k, a in enumerate(a_n):
102         p = 2**k
103         d_str = f"{float(D[k]):.10f}" if k >= 1 else "---"
104         r_str = f"{float(D[k-1] / D[k]):.6f}" if k >= 2 else "---"
105         print(f"{p:>5} -> {2*p:<4} {mp.nstr(a, 14):>16} {d_str:>14} {r_str:>10}")

```
